

Informe Tarea 1

Felipe Alvarez Paillahueque, falvarezp@usm.cl

Catalina Nolan Espinoza, cnolan@usm.cl

Martin Flores Silva, mfloress@usm.cl

Tomas Canobra Andrade, tcanobra@usm.cl

Rebeca Orellana Perasso, rorellanap@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

La función lectura de sensor tiene como propósito fundamental actuar como un generador de datos para el sistema, simulando el comportamiento de un sistema o sensor que capta señales del entorno. Para cumplir con este objetivo el sistema se rige por unas ciertas partes:

Rango de salida: Esta parte debe entregar valores enteros dentro del rango entre 0 y 255. Esta restricción técnica es debido a que el sensor entrega una salida de un convertidor analógico de 8 bits.

Simulación de magnitud: Los valores retornados por la función se interpretan en unidades de milivoltios[mV]. Esto permite que la simulación sea mucho más realista acorde a lo que se podría obtener.

Además el uso del rand() es vital ya que este es el encargado de darnos números 100% aleatorios e impredecibles o de manera de que se simulen los cambios reales de un sensor de este tipo.

Lógica del Algoritmo: El diseño es corto pero eficaz así se saca el mayor provecho al sistema haciendo una ejecución más rápida y eficiente. El flujo de datos consiste en:

1. Calcular un número con una división entre el número aleatorio y el 256(rand()%256), asegurando que el resultado nunca exceda los 8 bits de resolución.
2. El valor resultante es devuelto al programa principal o a la función controladora. Permitiendo compatibilidad con posibles cálculos de números mayores en un futuro.

Todo esto esta mejor graficado en un diagrama de flujo, como es el caso de la **figura 1**

1.2. Diseño completo de la nueva función main()

La función main() fue rediseñada para que sirva como el orquestador central del programa. Su objetivo principal no es procesar los datos directamente, sino que gestionar

el flujo de ejecución y coordinar las llamadas hacia las funciones especializadas que hacen el trabajo. El `main()` se divide en 3 pilares fundamentales:

1. Al iniciar, `main()` el responsable de preparar las condiciones necesarias para la simulación. Esto incluye declarar arreglos de sensores y luego así recibir los números aleatorios para que sea realista.
2. Ocupa `usarSensores()` para así no depender de bucles `for` y llamadas a `printf`, el nuevo `main()` delega toda la lógica operativa a la función `usarSensores()`. Esto reduce la complejidad visual del código y separa la interfaz del usuario. EL `main()` solo se preocupa de mandar la orden a través de el tipo de dato llamado Acción.
3. El ciclo de vida del flujo secuencial: La función implementa una secuencia lógica de estados que garantiza la integridad de los datos.
 - a. Fase de inicialización: Invoca el estado `INIT` para asegurarse que el arreglo de memoria comience en un estado conocido.
 - b. Fase de captura: Activa el estado `READ`, donde se prueba el arreglo con los datos simulados.
 - c. Fase de presentación: Finalmente, ejecuta el estado `SHOW` para entregar los resultados al usuario final.

Este diseño permite que el `main()` sea extremadamente ligero y fácil de leer. Al centralizar las operaciones en una sola función controladora, se facilitan las detecciones de error. Si se desea repetir el proceso de lectura solo se requiere añadir una línea adicional al `main()`, sin necesidad de reescribir la lógica de nuevo. En la **figura 2** se puede apreciar el diagrama de flujo de la función.

2. Parte 2

2.1. Hito 1

Responder todas las preguntas que se realizan en el enunciado:

Pregunta 1.-¿Qué es lo que retorna la función “`clock()`”?

- Retorna el tiempo de CPU consumido por el programa hasta el momento de la llamada, expresado en unidades de ciclos de reloj o “ticks”. En el código, se utiliza para capturar el estado del contador justo antes y justo después de la ejecución del ciclo “`for`”.

Pregunta 2.-¿Qué unidades de medida tienen las variables “`inicio`” y “`fin`”?

- Las unidades son ticks (pulsos de reloj del procesador). Es una unidad de medida interna del sistema que representa el tiempo de procesamiento dedicado al hilo actual.

Pregunta 3.-¿Por qué para obtener el tiempo total de duración del ciclo “`for`” se debe dividir la diferencia entre `fin` e `inicio` con el valor constante `CLOCKS_PER_SEC`?

- Se usa esta división para normalizar la medida, ya que la diferencia (`fin - inicio`) entrega el tiempo en ticks, dividirla por la constante `CLOCKS_PER_SEC` permite convertir esa magnitud a una unidad de tiempo estándar (segundos). Esto se hace mediante un proceso de conversión explícita.

Pregunta 4.-¿Explicar que hace `(double)` como prefijo (`fin - inicio`)?

Mediante una conversión explícita, se transforma el valor `int` (entero) resultante de la resta en un `double` (número de punto flotante) el cual entrega mayor precisión del resultado antes de operar con él.

Pregunta 5.-¿Explicar para qué sirve utilizar `(double)` como prefijo (`fin - inicio`)?

- Sirve para dar mayor precisión al cálculo. Debido a que en "C" si se dividen dos números enteros, el resultado se trunca y se pierden los decimales. Al convertir el numerador a "double", se fuerza al compilador a realizar una división decimal, lo cual, reiterando, entrega mayor precisión de los cálculos.

2.2. Hito 2

La función *`cicloPrueba(caso_t caso)`* fue diseñada como un módulo de medición de rendimiento. Su objetivo es calcular el tiempo promedio que tarda el procesador en ejecutar diferentes tipos de operaciones dentro de un ciclo repetitivo, dependiendo del valor recibido en el parámetro `caso`.

Para esto primero se utilizó una estructura de tipo *`enum caso_t`* con las variables `VACIO`, `ESPACIO`, `ASTERISCO` Y `NEWLINE` dentro para así facilitar el uso de estas en un condicional (`if` o `switch`)

Posteriormente se asigna un único parámetro a la función (*`caso`*) para saber qué acción se ejecutara dentro del ciclo. De esta forma la función se convierte en un módulo flexible, ya que depende del parámetro recibido.

Dentro de la función incluye la medición del tiempo, esto siendo posible por el comando `clock()`, que registra el tiempo de inicio y al final de la función registra el tiempo de finalización. Al tener ambos registros de tiempo realiza una diferencia para conseguir el tiempo total y realizar una división por la cantidad de iteraciones para que nos entregue el tiempo promedio que se tomó en realizar el ciclo.

Por último, también se encuentra toda la estructura del ciclo, en la que se evaluarán los casos dados como parámetros de la función que tienen como funcionamiento una serie de condicionales para ejecutar distintas instrucciones, como por ejemplo imprimir un asterisco o un espacio, lo cual se puede apreciar en el diagrama de flujo de la función en la **figura 3**.

2.3. Hito 3

El propósito fundamental de las mediciones es observar el cuello de botella que representa el sistema de entradas y salida (E/S) frente al uso de la memoria del sistema. En simples palabras, hacer notar que un cálculo puede ser súper fácil pero es costoso en temas de tiempo.

- Caso nada: El promedio era de 1[ns] indica que el procesador ejecuta el ciclo casi instantáneamente cuando no tiene que comunicarse con el exterior.
- Costo de visualización: Los demás casos espacio, asterisco, newline, muestran promedios superiores a los 28,000[ns]. Esto se debe a que cada printf requiere que el programa se detenga, e pida permiso al sistema operativo y espera a que el terminal dibuje el carácter.
- Desviación estándar: Este dato refleja que el tiempo de respuesta de la terminal no es constante, ya que depende de otros procesos de el computador que se estén ejecutando en ese instante en el que ocurre el proceso.

El sistema es tan rápido como su componente más lento, o sea que, aunque pueda ir más rápido, el proceso más lento va a ser el que lleve el movimiento dentro. El problema no es el código ni el procesador, sino que la interfaz (terminal) que se demora en mostrar cada uno de los signos de texto.

Caso	tprom1	tprom2	tprom3	tprom4	tprom5	tprom6	tprom7	tprom8	tprom9	tprom10	Promedio	Desv. Estandar
NADA	1	1	1	1	1	1	1	1	1	1	1	0
ESPACIO	30028	41263	32048	41943	33208	42114	32506	32036	31429	45535	36211	5465,932254
ASTERISCO	29392	40302	31393	41598	32380	40597	31325	30300	30240	40555	34808,2	4929,89967
NEWLINE	31625	36300	27717	35879	28120	35611	28247	26469	26627	5223	28181,8	8514,002535

3. Anexo A:

A continuación se presentan algunos diagramas de flujos de los códigos usados en el informe para complementar la explicación del diseño.

Diagrama de flujo - funcion lecturaSensor()

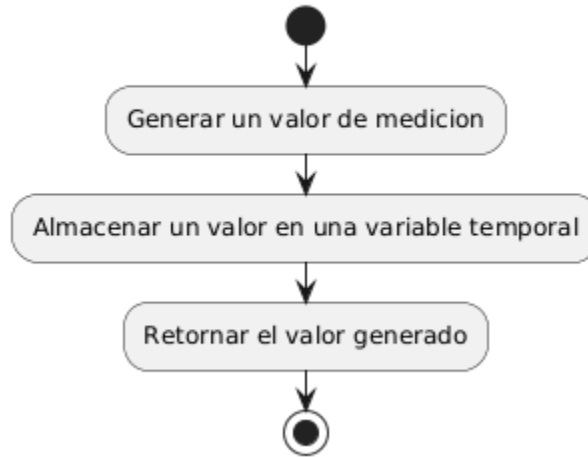


Figura 1

Diagrama de flujo - funcion main()

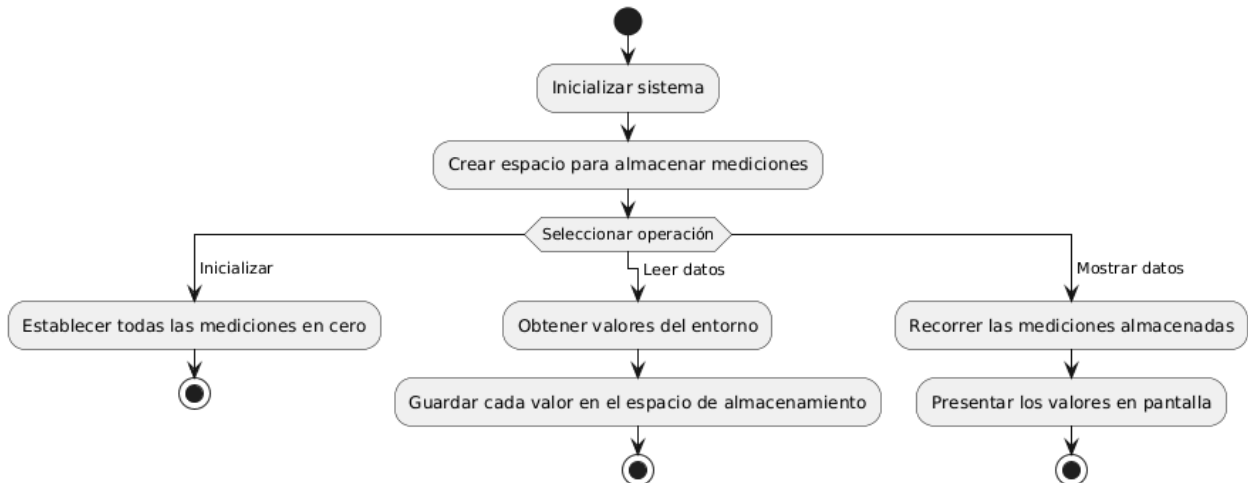


Figura 2

Diagrama de flujo - funcion cicloPrueba()

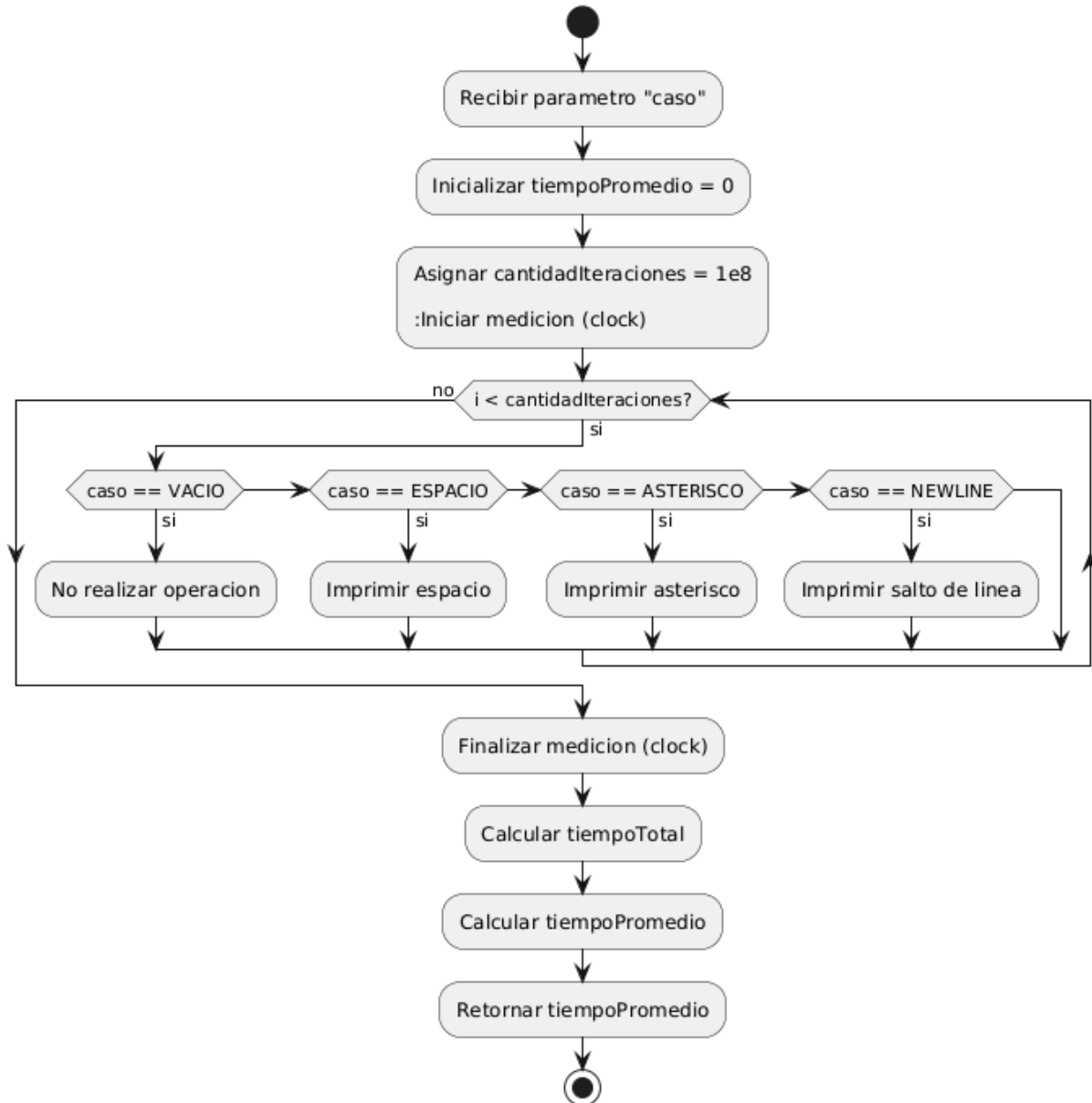


figura 3